CSC-223
Assignment 0
Work in a Pair

Utility Functions

In this assignment, you will work in pairs to develop a collection of reusable utility methods in C# that demonstrate fundamental programming concepts.

What You Need to Do: Adding to the Project Structure

As shown in Figure 1, our utility code will be in the `Utilities` folder and tests in the corresponding `Utilities.Tests` folder.

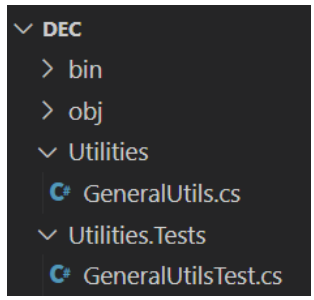

Figure 1: The desired project structure including utility code and tests.

What You Need to Do: Method Implementations

Utility functions serve as the fundamental building blocks of programming—small, self-contained methods that perform specific tasks and can be reused throughout an application. Through this assignment, you will gain practical experience with C# syntax while learning to write clean, focused methods with a single responsibility. You will practice proper parameter validation, exception handling, and develop skills in creating reusable, maintainable code that applies foundational programming concepts in practical contexts. A well-designed utility function performs a single, clearly defined task with minimal dependencies, making it easily reusable across different scenarios. Such functions should include appropriate input validation, thorough documentation, and follow consistent naming conventions. By focusing on these principles, you'll develop the skills necessary to write efficient, reliable code that forms the backbone of larger applications.

Table 1: Methods to be implemented in the `GeneralUtils.cs` file.

Return Type	Method	Arguments	Description
`bool`	`Contains<T>`	`T[] array, T item`	Checks if an array contains a specific item using equality comparison: `EqualityComparer<T>`.
`string`	`GetIndentation`	`int level`	Returns a string with spaces for the specified indentation level: 4 spaces per level.
`bool`	`IsValidVariable`	`string name`	Checks if a string contains only lowercase letters.
`bool`	`IsValidOperator`	`string op`	Checks if a string is a valid operator (`+`, `-`, `*`, `/`, `//`, `%`, `**`).
`int`	`CountOccurrences`	`string s, char c`	Counts how many times a given character appears in a string.

`string`	`ToCamelCase`	`string s`	Converts space-separated words to *camelCase* format. That is, the first character is lower case, but all subsequent spaces are removed and the next letter is capitalized. For example, `"Hello world test string" → "helloWorldTestString"`
`bool`	`IsPasswordStrong`	`string pwd`	Checks if a password meets strength requirements. A strong password must be at least 8 characters long and contain at least one lowercase letter, one uppercase letter, one digit, and one special character (any non-alphanumeric character).
`List<T>`	`GetUniqueItems<T>`	`List<T> list`	Returns another list with only unique items from the given list; the given list should not be modified. Throws `ArgumentException` if list is `null`.
`double`	`CalculateAverage`	`int[] numbers`	Calculates the average value of an array of integers. Throws `ArgumentException` if array is `null` or empty.
`T[]`	`Duplicates<T>`	`T[] array`	Returns the set of all items from an array that are duplicated more than one time. It is reasonable to use a dictionary in an implementation.

What You Need to Do: Testing

All tests should be xUnit tests and adhere to the norms of `xUnit` including using `[Fact]` and `[Theory]`.

New to CSC-223! LLMs can be helpful tools for generating testing code for your assignments. When using an LLM to create tests, include prompts like "write thorough tests for all methods", "make sure to test with duplicate data as well" to ensure comprehensive coverage, and "use `[Theory]` and `InlineData` to avoid redundancy in testing code". ***However, be warned that generated code is not always perfect—your professor has encountered several cases where tests were written incorrectly for the corresponding code.*** Always review each test carefully to verify it is testing what you intend and functioning properly with your implementation. It is also fair to ask the LLM to see if any cases have been missed in the tests that it generates. In fact, it is advised that you ask an LLM to 'check its work'; do not accept a first answer.

Even though you may use an LLM for testing, you will explicitly be asked about test coverage.

What You Need to Do: Commenting

Your code should be well documented, including XML documentation comments for methods, blocks of code, and header comments in each file. `xunit` tests should be clear enough that comments are generally not required by a reader.

Header Comments

Your program must use the following standard comment at the top of *each source code file*. Copy and paste this comment and modify it accordingly with your files.

```
/**
 * Write a succinct, meaningful description of the class here. You should avoid wordiness
 * and redundancy.
 *
 * Bugs: (a list of bugs and / or other problems)
 *
```

```
    * @author <your name>
    * @date   <date of completion>
    */
```

XML Documentation Comments

Comment your code with a *reasonable amount of comments* throughout the program. Each non-obvious method should have an XML comment that includes information about input, output, overall operation of the function, as well as any limitations that might raise exceptions. As an example:

```
/// <summary>
/// Calculates the average of a set of numbers.
/// </summary>
/// <param name="numbers">An array of integers to calculate the average from</param>
/// <returns>The average as a double</returns>
/// <exception cref="ArgumentException">Thrown when the array is null or
empty</exception>
public double CalculateAverage(int[] numbers)
{
    // Implementation
}
```

Inline Comments

Each *block* of code (3-4 or more lines in sequence) in a function should be commented. Comments go above and before the code they are referencing.

It is ***prohibited*** to use *long* comments to the right of lines of source code; attempt 80 character-wide text in source code files. Succinct comments (a few words at maximum can be fine).

Region Documentation

Consider using `#region` directives to organize your code. Make sure to include a brief comment explaining the purpose of each region.

```
#region <Brief Comment>

// Methods that provide support functionality

#endregion
```

Submitting: Source Code

You will demonstrate your source code to the instructor, in person. Be ready to demonstrate successful execution of all unit tests and describe all tests and code you were required to implement.